

Chapter 1. 에이전트의 최소 구성 요소

이 챕터에서 배우는 것

- "AI 에이전트"라는 말이 실제 코드에서는 무엇을 가리키는지
- Book-forge의 `LLM` Protocol이 왜 OpenAI/Anthropic/Ollama를 똑같은 인터페이스로 감싸는지
- 에이전트를 최소로 정의하면 무엇이 남고 무엇이 빠지는지

이런 분이 먼저 읽으면 좋습니다: "에이전트"라는 단어를 여러 문서에서 다르게 쓰는 걸 보고 혼란스러웠던 분. 이 챕터는 정의를 논쟁하는 대신, 실제로 동작하는 코드 하나를 기준으로 삼는다.

1.1 가장 작은 정의부터 시작한다

"AI 에이전트"라는 말은 문맥마다 다른 것을 가리킨다 — 어떤 글은 자율적으로 도구를 골라 쓰는 시스템을 뜻하고, 어떤 글은 단순히 "LLM을 호출하는 함수"를 그렇게 부른다. 이 책은 논쟁에 끼어드는 대신 Book-forge가 실제로 코드로 구현한 정의에서 출발한다 — `src/book_forge/llm/provider.py`가 그 출발점이다.

```
@runtime_checkable
class LLM(Protocol):
    """모든 provider 구현체가 만족해야 하는 최소 인터페이스."""

    model: str

    def generate(
        self, prompt: str, *, system: Optional[str] = None, max_tokens: int = 4
    ) -> str: ...
```

이게 전부다 — 문자열 하나(`prompt`)를 받아 문자열 하나(`generate()`)의 반환값)를 돌려주는 것. Book-forge의 모든 에이전트는 이 인터페이스 위에 얹게 쌓은 껍데기일 뿐이다. "에이전트가 무엇을 하는가"는 이 `generate()` 호출 앞뒤에 무엇을 붙이느냐로 결정된다 — 어떤 프롬프트를 조립해 넘기는지, 응답을 어떻게 파싱하는지, 그 결과로 무엇을 하는지.

1.2 세 개의 구현체, 하나의 계약

LLM Protocol을 실제로 만족하는 구현체는 세 개다.

구현체	실제로 호출하는 것	기본 모델
<code>OpenAILLM</code>	OpenAI Chat Completions API	<code>gpt-5-nano</code>
<code>AnthropicLLM</code>	Anthropic Messages API	<code>claude-haiku-4-5-20251001</code>
<code>OllamaLLM</code>	로컬 <code>http://localhost:11434/api/generate</code>	<code>llama3.2</code>

세 클래스의 `generate()` 시그니처는 동일하지만, 내부에서 실제로 하는 일은 서로 다른 HTTP 요청 형식을 조립하는 것이다. 예를 들어 `OllamaLLM.generate()` 는 이렇게 페이로드를 만든다.

```
payload = {
    "model": self.model,
    "prompt": prompt,
    "system": system or "",
    "stream": False,
    "think": False, # 추론 모델이 "생각"만 하고 답을 안 내는 문제 방지
    "options": {"num_predict": max_tokens},
}
response = requests.post(f"{self._base_url}/api/generate", json=payload, timeout=180)
```

`think: False` 옵션 하나가 이 책 전체에서 반복해 등장할 "에이전트는 왜 실패하는가"(3장)의 첫 사례다 — `qwen3.6:35b-mlx` 같은 추론(thinking) 모델은 사고 과정을 별도 필드에 담고 최종 응답(`response`)은 비워둘 수 있다. `num_predict` (토큰 예산)를 추론에 다 써버리면 챗터 파일이 통째로 빈 채 저장되는 사고가 실제로 있었다

(`provider.py` 의 주석에 이 실측 경위가 그대로 남아 있다). `think: False` 는 그 사고 과정을 건너뛰고 바로 답을 채우게 강제하는, 코드 한 줄로 막은 실패 사례다.

1.3 provider 선택 — 환경변수 하나가 전체 파이프라인을 바꾼다

`create_llm()` 팩토리 함수가 어떤 구현체를 만들지 결정한다.

```
def create_llm(provider: Optional[str] = None, model: Optional[str] = None) -> LLM:
    resolved = (provider or os.environ.get("LLM_PROVIDER") or "ollama").lower()
    ...
```

우선순위는 명시적 인자 → `LLM_PROVIDER` 환경변수 → 기본값 `"ollama"` 순이다. 기본값을 클라우드가 아니라 로컬로 둔 이유는 코드 주석에 명시돼 있다 — "API 키 없이 바로 동작해 오프라인/로컬 개발을 1급 경로로 만든다." 이 한 줄의 설계 결정이 Book-forge 전체의 성격을 규정한다 — 이 책의 모든 실측 예제도 API 키 없이 로컬 Ollama로 재현 가능하다.

 **개발자 TIP:** 에이전트를 새로 만들 때 "이 함수가 Protocol을 만족하는가"만 확인하면 provider 걱정 없이 코드를 짤 수 있다.

`planner.py` · `chapter_drafter.py` · `chat_agent.py` 어디에도 `if provider == "openai"` 같은 분기가 없다 — 전부 `llm.generate(...)` 한 줄만 호출한다.

1.4 이 정의에 일부러 넣지 않은 것

이 최소 정의에는 "도구를 자율적으로 선택한다"거나 "여러 단계를 스스로 계획한다"는 요건이 없다. Book-forge의 개별 에이전트(`propose_plan()` , `design_toc()` 등) 는 정확히 한 번 `generate()` 를 호출하고 끝난다 — 도구 호출도, 반복 루프도 없다. 그런데도 이 책은 이들을 "에이전트"라 부른다. 왜인가 — 다음 장에서 확인하겠지만, 이 함수들을 실제 시스템으로 만드는 것은 LLM 호출 자체가 아니라 **그 호출 앞뒤에 계속** (`@agent_eval`)이 붙어 있다는 사실이다. "에이전트"와 "그냥 LLM을 부르는 함수"를 가르는 경계가 바로 이 책의 3부·4부가 다루는 Harness Engineering이다.

이 챕터의 핵심

- 에이전트의 최소 계약은 `prompt: str → str` 이다. Book-forge의 `LLM` Protocol이 이를 코드로 명시한다.
- `provider(OpenAI/Anthropic/Ollama)`는 껍데기일 뿐, 에이전트 코드는 `provider`를 몰라도 된다. `create_llm()` 이 환경변수로 그 결정을 한 곳에 모은다.
- 작은 구현 디테일(`think: False`)이 실제 장애로 이어진 사례가 이미 있다. 에이전트를 다룰 때 "모델이 답을 냈는가"를 당연하게 여기면 안 된다.

참고 자료

- `src/book_forge/llm/provider.py` — `LLM` Protocol과 3개 구현체 전체
- `src/book_forge/config.py` — `load_config()` 가 `.env` 를 읽어 `LLM_PROVIDER` 등을 `os.environ` 에 채우는 경로

다음 챕터는 이 `generate()` 호출 하나가 실제 에이전트(`PlannerAgent`) 안에서 어떻게 프롬프트로 조립되고, 응답이 어떻게 파싱되며, `@agent_eval` 계측이 어느 지점에 끼어드는지 한 줄씩 따라간다.